

Neuromorphic Workload Acceleration and Architectural Efficiency Analysis

Parallel Simulation of a Spiking Neural Network: Sequential, OpenMP, and OpenCL Implementations on

a Commodity Laptop and an HPC Compute Node

Abiye Girma

GSR/2914/18

Parallel Computing Project

Abstract—This report presents the design, parallelization, and architectural evaluation of a Leaky Integrate-and-Fire (LIF) Spiking Neural Network (SNN) simulator implemented in three forms: a single-threaded sequential C++ baseline, a multi-threaded OpenMP CPU version, and an OpenCL GPU/accelerator version. The workload uses a sparse, randomly generated synaptic connectivity matrix stored in Compressed Sparse Row (CSR) format to model irregular, brain-inspired connectivity. The three implementations were benchmarked on two platforms of very different scale: an HP Pavilion laptop (Intel Core i5-7200U, 2 physical cores) and a 40-core/80-thread Intel Xeon E5-2698 v4 HPC compute node equipped with an NVIDIA Tesla V100-SXM2-16GB accelerator. Correctness was verified by confirming identical spike counts (14 spikes) across all three versions on both platforms. On the HPC node, the OpenMP version achieved a peak strong-scaling speedup of 8.13x at 16 threads (50.8% efficiency), the performance ceiling of this implementation, before degrading at higher thread counts due to memory-bandwidth contention. On the laptop, by contrast, both parallel versions were slower than the sequential baseline at the fixed comparison workload (OpenMP 0.94x, OpenCL 0.42x), demonstrating that on a 2-core device, thread-management and accelerator-launch overhead exceeds any benefit from parallel execution. The OpenCL version underperformed OpenMP on both platforms and was even slower than single-threaded sequential execution on the laptop, evidence that per-timestep host-device synchronization overhead, not compute throughput, governs its performance at this problem scale. A Roofline analysis confirms that the workload is strongly memory-bound on both platforms, with measured performance reaching only 16.6-17.8% of the memory bandwidth ceiling on the laptop and 2.5-5.7% on the HPC node, supporting the central architectural thesis that the irregular, sparse, scatter-heavy memory access pattern of spiking neural networks is the dominant bottleneck on von Neumann (CPU/GPU) hardware and the primary motivation for dedicated neuromorphic hardware such as Intel Loihi and IBM TrueNorth.

I. INTRODUCTION

Neuromorphic computing aims to emulate the event-driven, massively parallel, and energy-efficient information processing of biological neural tissue. Spiking Neural Networks (SNNs) are the principal computational abstraction used to study this class of architecture: rather than propagating continuous activations as in conventional artificial neural networks, SNNs communicate via discrete, asynchronous spikes routed through a sparse synaptic connectivity graph [1]. Simulating

SNNs at scale on conventional hardware is computationally demanding precisely because the workload characteristics, sparse connectivity, irregular memory access, and fine-grained, asynchronous update dependencies, are poorly matched to the regular, vectorizable access patterns that CPUs and GPUs are optimized for.

This project implements a LIF-based SNN simulator and evaluates it across three execution models, sequential CPU, multi-threaded CPU (OpenMP), and accelerator (OpenCL), on two platforms representing opposite ends of the available-parallelism spectrum: a 2-core consumer laptop and a 40-core/80-thread HPC node with GPU acceleration. The underlying architectural concern is the von Neumann bottleneck: because conventional CPUs and GPUs keep their processing units and their memory physically separate, any operation whose target address cannot be predicted in advance, such as the scattered write to a postsynaptic neuron's potential triggered by an arbitrary firing pattern, pays the full latency of a round trip across the memory hierarchy rather than being resolved locally, no matter how much raw compute throughput the device has. The objective of this project is not merely to demonstrate parallel speedup, but to use the gap between achieved and theoretical-peak performance to make an evidence-based architectural argument about why specialized neuromorphic hardware (e.g., Intel Loihi [2], IBM TrueNorth [3]) exists, and what bottleneck it is specifically designed to remove.

II. METHODOLOGY

A. Neuron Model

Each neuron follows the Leaky Integrate-and-Fire model. At every 1 ms timestep, the membrane potential V of every neuron decays toward a resting potential of 0 mV with a fixed decay factor of 0.9512 (corresponding to a membrane time constant of 20 ms). A neuron spikes when V exceeds a threshold of 1.0 mV, after which V is reset to 0 mV. The first 10% of neurons in the network additionally receive independent Poisson-distributed input spikes at a configurable input rate (20 Hz by default), which drives activity in the otherwise quiescent network. All three implementations share

this identical parameter set (`types.hpp`), and all use the same fixed random seed (42) for connectivity and input generation, which is what permits a direct, deterministic correctness comparison across implementations.

B. Sparse Connectivity Representation

Synaptic connectivity is generated as a random directed graph with a configurable connection density and stored in Compressed Sparse Row (CSR) format (`sparse_matrix.hpp`): a `row_ptr` array of length $N+1$, a `col_idx` array giving the postsynaptic target of each synapse, and a `values` array giving synaptic weight. This is the standard sparse-matrix layout for large, irregular graphs and is what allows the propagate phase, in which a firing neuron must add its weight to every postsynaptic target, to be expressed as a single pass over only the non-zero synapses rather than a dense $N \times N$ pass. The CSR matrix is built in a single pass to avoid a two-stage random-number-generator inconsistency that was identified and corrected during development.

C. Sequential Baseline

The sequential version (`snn_sequential.cpp`) executes four phases per timestep, in order: (1) Poisson input generation, (2) leak/decay of all membrane potentials, (3) propagate, in which every neuron that spiked in the previous timestep walks its CSR row and accumulates its weight into each postsynaptic neuron’s potential, and (4) threshold check and reset. Using the previous timestep’s spikes for propagation (a synchronous update scheme) avoids any read-after-write hazard and gives an unambiguous, race-free reference output used to validate both parallel versions.

D. OpenMP CPU Parallelization

The architectural challenge in parallelizing the propagate phase is that multiple threads processing different source neurons can attempt to update the same postsynaptic target concurrently, a direct instance of the synaptic-divergence/load-imbalance problem inherent to irregularly connected, event-driven networks. The OpenMP implementation (`snn_openmp.cpp`) avoids this without atomics or locks by giving each thread a private, cache-line-padded local copy of the potential array. Each thread accumulates only into its own buffer during the propagate phase (no cross-thread writes are possible), after which a separate parallel reduction merges all per-thread buffers into the shared potential array. Padding each thread’s buffer to a 64-byte cache-line boundary additionally prevents false sharing between adjacent threads’ buffers. The leak and threshold-check phases are embarrassingly parallel and are distributed with `#pragma omp for`. Across the whole timestep loop the implementation uses six OpenMP barriers (down from an earlier, unoptimized fifteen identified during development), corresponding to the minimum synchronization required for correctness: after input generation, after leak, after propagate, after the buffer merge, after threshold check, and after the spike-array swap.

Compiler flags used for each build (e.g., optimization level, `-fopenmp`, architecture-specific vectorization) were not captured in the available build logs and are not claimed here; this is noted explicitly as a limitation in Section 5, since differing optimization levels between the sequential and OpenMP binaries could materially affect the baseline comparison used throughout Section IV (for example, the HPC node’s 1-thread OpenMP run measuring faster than its sequential baseline, discussed in Section IV-B).

E. OpenCL Accelerator Port

The OpenCL implementation (`snn_opencl.cpp`) maps the same three compute phases (leak, propagate, spike-check) onto three separate kernels (`leak_kernel`, `propagate_kernel`, `spike_kernel`), dispatched once per timestep with a configurable work-group size. The propagate kernel is launched with one work-item per non-zero synapse (CSR triplet), using a parallel `src_idx` array generated alongside `row_ptr/col_idx` so that each work-item can resolve its source and destination neuron independently. Because the device-side kernel source file (`kernels.cl`) was not provided for this report, this analysis is restricted to the host-side memory and synchronization behavior, which is directly evidenced by the code; the specific in-kernel conflict-resolution strategy used to prevent races on the shared destination potential array (atomic add, replicate-and-reduce, or another scheme) could not be verified and is flagged as a limitation in Section V. What is verifiable from the host code is that the implementation performs a fully synchronous round-trip every timestep: the merged spike array is written to the device with a blocking `clEnqueueWriteBuffer`, all three kernels are launched and explicitly waited on with `clWaitForEvents` for profiling, and the resulting spike array is read back with a blocking `clEnqueueReadBuffer` before the next timestep’s input can be merged in on the host. This per-timestep host-device transfer and synchronization is, as discussed in Section IV-D, the most plausible explanation for the OpenCL version’s underperformance relative to OpenMP on both platforms.

Which physical device executed OpenCL on the laptop (the CPU via Intel’s OpenCL runtime, or the integrated HD Graphics 620) was not confirmed from the available run logs, since the laptop hardware table lists no discrete GPU and the device-selection print statement’s output for that run was not captured; this is treated as an open question rather than assumed, and is listed as a limitation in Section V. The metric used for synaptic throughput also differs between implementations and is made explicit here rather than left implicit: the sequential and OpenMP versions increment an exact per-edge counter every time the propagate loop actually visits a synapse, whereas the OpenCL host code (visible in the `simulate()` method above) computes `total_synaptic_ops` as `total_spikes` multiplied by the network’s average out-degree, an estimate rather than an exact count. Because individual neurons’ out-degrees vary around that average, the two metrics are not directly

TABLE I: Hardware Specifications

Specification	HP Pavilion Laptop	HPC Compute Node
CPU model	Intel Core i5-7200U @ 2.50 GHz	Intel Xeon E5-2698 v4 @ 2.20 GHz
Physical cores	2	40
Logical threads	4	80
L1-D cache	64 KB (32 KB/core)	64 KB (32 KB/core)
L2 cache	512 KB (256 KB/core)	512 KB (256 KB/core)
L3 cache (shared)	3 MB	50 MB
Accelerator	None (CPU only)	NVIDIA Tesla V100-SXM2-16GB (OpenCL)
Operating system	Windows 11	Linux
Compiler	MSVC / g++	g++
Memory bandwidth used in Roofline model	25.6 GB/s (DDR4-3200, single channel; platform default)	76.8 GB/s (user-supplied estimate for the compute node)
Peak compute used in Roofline model	120 GFLOPS (platform default estimate)	1050 GFLOPS (user-supplied estimate for the compute node)

comparable, particularly when total spike counts are small; this is the source of the synaptic-operation-count differences discussed in Sections IV-A and IV-D, and is not evidence that the OpenCL kernel skips synapses.

III. EXPERIMENTAL SETUP

Two hardware platforms were used to capture both small-scale and large-scale parallel behavior.

Note on the Roofline inputs: the laptop’s bandwidth/compute figures are the script’s built-in conservative defaults for a single-channel DDR4-3200 mobile CPU; they were not independently re-measured with a memory benchmark (e.g., STREAM) and should be treated as estimates. The HPC node’s figures (76.8 GB/s, 1050 GFLOPS) were supplied directly at the command line for this run; exact RAM channel configuration for the node was not confirmed, and these values are closer to a multi-core AVX2 CPU estimate than to the V100 accelerator’s published HBM2 bandwidth (~ 900 GB/s) and compute throughput, since the OpenCL device’s standalone memory bandwidth was not independently benchmarked in this study. The Roofline conclusions in Section IV-F should therefore be read as platform-level estimates rather than device-certified figures.

All experiments use the parameter sweep defined in `run_all_experiments.py`. Two distinct workload sizes were used because the laptop’s limited memory and core count make the HPC-scale problem impractical: the laptop sweep uses a 5,000-neuron network at 5% connection density for strong scaling and version comparison, while the HPC sweep uses a 10,000-neuron network at 2% density for the same experiments (kept low to keep total synapse count, and therefore memory footprint, comparable across the two very different node sizes). Strong-scaling experiments fix the problem size and vary the thread count (1 to 8 threads on the laptop’s 4 logical threads; 1 to 64 on the HPC node’s 80 logical threads). Weak-scaling experiments grow the network size with the square root of the thread count to hold per-thread synaptic work approximately constant, since synapse count scales with N^2 . The OpenCL work-group sweep and the Roofline varying-size sweep are run identically on both platforms, scaled to each platform’s practical problem size.

IV. RESULTS AND ANALYSIS

A. Correctness Verification

Both platforms produced an identical spike count of 14 across all three implementations (Sequential, OpenMP, OpenCL) for the correctness workload (1,000 neurons, 100 timesteps, 10% density, seed 42), which the benchmark harness flags as PASS (agreement within 1%). This confirms that the parallel and accelerated implementations are computing the same LIF dynamics as the sequential reference, not merely running faster. The raw timings on this very small workload (HPC: Seq 0.3 ms, OpenMP 1.71 ms, OpenCL 80.0 ms) already foreshadow the trend explored below: OpenMP at 4 threads is nearly 6x slower than sequential on this tiny workload, showing that thread-management overhead alone exceeds any parallel benefit until the problem is large enough to amortize it, and OpenCL’s 80.0 ms reflects fixed device-initialization and per-timestep launch costs rather than any actual compute bottleneck.

The HPC correctness.csv also reports a large difference in the synaptic_ops field between the exact-counting versions and OpenCL (Sequential/OpenMP: 22,349; OpenCL: 1,401). This is fully explained by the metric-definition difference described in Section II-E: OpenCL’s reported figure is total_spikes (14) multiplied by the network’s average out-degree (approx. 100 for 1,000 neurons at 10% density), an estimate that diverges sharply from the exact count at this low a spike count, rather than evidence of a counting bug or an incomplete kernel sweep.

B. Strong Scaling

Strong scaling fixes the problem size and increases thread count. Results differ substantially between platforms because of the very different core counts and memory subsystems available.

Laptop sequential reference: 509.48 ms. HPC sequential reference: 3452.59 ms (used for the speedup ratios in the table). The 1-thread OpenMP run was faster than the separately compiled sequential baseline, likely due to differing compiler optimization flags, resulting in an efficiency slightly above 100%.

On the laptop, with only 2 physical cores and 4 logical threads, scaling collapses immediately beyond 2 threads (efficiency falls from 71.6% at 2 threads to 11.2% at 8 threads) for two compounding reasons: at 4 threads the machine is at its logical-thread limit, so each additional thread shares

TABLE II: Strong Scaling Results

Threads	Laptop Time (ms)	Laptop Speedup	Laptop Efficiency	HPC Time (ms)	HPC Speedup	HPC Efficiency
1	512.0	1.00x	99.5%	3113.7	1.11x	110.9%
2	355.6	1.43x	71.6%	1727.4	2.00x	99.9%
4	405.1	1.26x	31.4%	1075.2	3.21x	80.3%
8	567.2	0.90x	11.2%	653.7	5.28x	66.0%
16	—	—	—	424.4	8.13x	50.8%
32	—	—	—	515.9	6.69x	20.9%
64	—	—	—	655.9	5.26x	8.2%

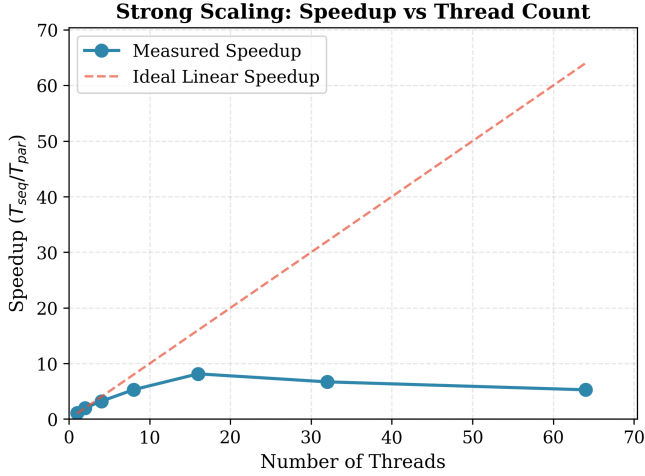


Fig. 1: Measured speedup vs. ideal linear speedup, HPC node, 1-64 threads.

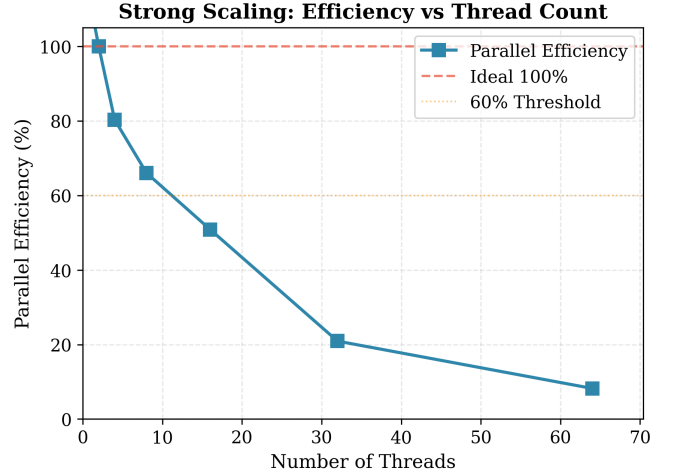


Fig. 2: Parallel efficiency vs. thread count, HPC node, with 60% reference line.

an execution unit via hyperthreading with no added compute capacity; at 8 threads the workload oversubscribes even the logical thread count, pushing the OS into active scheduling contention where threads are competing for CPU time rather than merely sharing execution ports, which explains why degradation continues past the 4-thread point rather than plateauing. On the HPC node, scaling is much healthier at low thread counts, reaching a peak speedup of 8.13x at 16 threads (50.8% efficiency) before reversing: at 32 and 64 threads, both wall-clock time and efficiency get worse, not just diminish (515.9 ms and 655.9 ms, vs. 424.4 ms at 16 threads). Because the propagate phase is memory-scatter-bound (Section IV-F), once enough threads are active to saturate the node’s shared memory bandwidth, adding further threads only increases cache and memory-controller contention without adding useful throughput; this indicates that irregular synaptic memory access, not core count, is the binding constraint for this workload.

The HPC node’s 1-thread OpenMP row reports 110.9% efficiency, meaning the 1-thread OpenMP build (3,113.7 ms) ran faster than the separately compiled sequential binary (3,452.59 ms) on the identical workload. Because exact compiler flags for each binary were not captured (Section II-D), the most likely explanation is a difference in compiler optimization or auto-vectorization between the two builds rather than any property of the parallel algorithm itself; this is listed as an open item in Section V rather than resolved here.

C. Weak Scaling

Weak scaling grows the network size with the square root of thread count so that synaptic work per thread is approximately

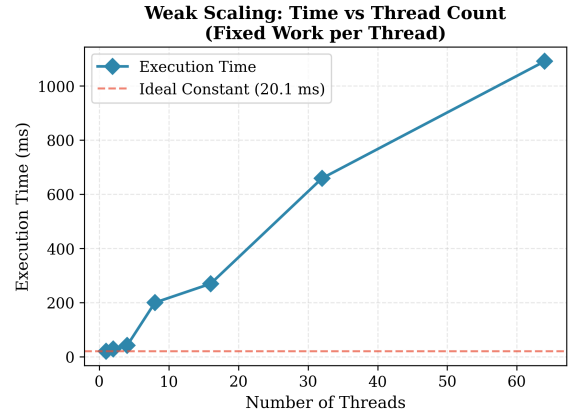


Fig. 3: Execution time vs. thread count under fixed work-per-thread scaling, HPC node, with ideal-constant reference line.

constant (synapse count scales as N^2). Ideal weak scaling would hold execution time flat.

Weak-scaling efficiency falls sharply on both platforms, far faster than in the strong-scaling case. Part of this drop is overhead rather than memory effects: the laptop’s 1-thread weak-scaling efficiency is only 73.5%, because even with no parallel work to distribute, the 1-thread OpenMP run (3.9 ms) is already slower than the sequential baseline (2.85 ms) at the same problem size, purely from barrier and buffer-allocation overhead. Beyond 1 thread, the synapse count’s quadratic growth with neuron count means doubling the network to keep per-thread work nominally constant in fact more than doubles the working-set size and the irregularity of the scatter pattern in $V[col_idx]$, so memory traffic grows faster than the formula intends.

TABLE III: Weak Scaling Results

Threads	Laptop Neurons	Laptop Time (ms)	Laptop Weak Eff.	HPC Neurons	HPC Time (ms)	HPC Weak Eff.
1	500	3.9	73.5%	2,000	20.1	100.5%
2	707	82.7	3.4%	2,828	28.8	70.2%
4	1,000	155.3	1.8%	4,000	40.9	49.5%
8	1,414	284.2	1.0%	5,656	199.6	10.1%
16	—	—	—	8,000	269.1	7.5%
32	—	—	—	11,313	658.3	3.1%
64	—	—	—	16,000	1,090.2	1.9%

TABLE IV: Version Comparison

Version	Laptop Time (ms)	Laptop Speedup	HPC Time (ms)	HPC Speedup
Sequential	527.53	1.00x	3,077.64	1.00x
OpenMP (8 threads)	560.61	0.94x	631.98	4.87x
OpenCL (GPU/accelerator)	1,255.04	0.42x	1,952.38	1.58x

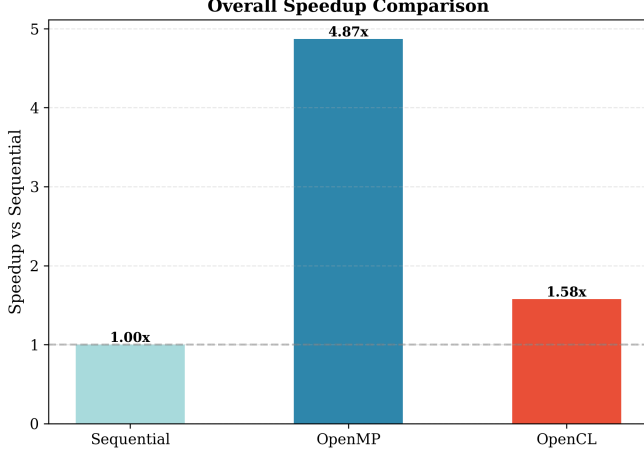


Fig. 4: Speedup of each version relative to sequential, HPC node.

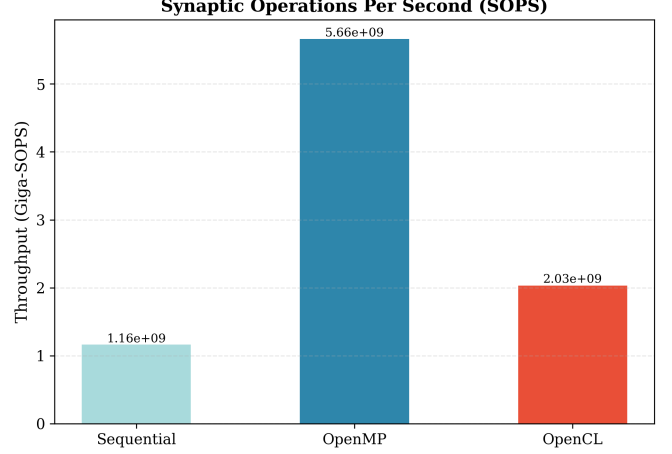


Fig. 5: Synaptic Operations Per Second for each version, HPC node.

The HPC weak-scaling data beyond 4 threads should be treated with caution rather than read at face value: total spike count jumps from 802 spikes at 4 threads (4,000 neurons) to 11,017,678 spikes at 8 threads (5,656 neurons), a roughly 14,000x increase for a 41% increase in network size. That magnitude of jump is not consistent with stable leak/threshold dynamics and indicates the network entered a runaway excitation regime at that size, rather than continuing to exercise the same dynamical behavior at a larger scale. Because the weak-scaling efficiency figures at 8, 16, 32, and 64 threads are computed from a network that is no longer behaving like the smaller ones, those efficiency numbers conflate two different effects, genuine memory-bandwidth saturation and a probable network-instability artifact, and should not be cited as a clean memory-wall measurement past the 4-thread point without first confirming (and, if necessary, correcting) the cause of the spike-count jump.

Weak efficiency on the HPC node drops from 100.5% at 1 thread to 1.9% at 64 threads, a degradation curve far steeper than anything in the strong-scaling results. The lower-thread-count portion of this trend (1-4 threads, where spike counts remain in the tens-to-hundreds range and behave consistently with the correctness and strong-scaling workloads) is read as genuine evidence for the memory-wall thesis; the higher-thread-count portion is reported for completeness but flagged as confounded, per the preceding paragraph.

D. Version Comparison: Sequential vs. OpenMP vs. OpenCL

At this fixed problem size, OpenMP delivers the best HPC result (4.87x) and throughput of 5.66 Giga-SOPS, more than 4.8x the sequential throughput (1.16 Giga-SOPS). OpenCL trails both: 1.58x speedup and 2.03 Giga-SOPS on the HPC node, and is actually slower than sequential on the laptop (0.42x), where it has no real GPU to exploit and is paying full OpenCL initialization and per-timestep synchronization overhead on top of CPU-only execution. On the HPC node, where an actual GPU is available, OpenCL is still 3.1x slower than OpenMP. The most direct explanation, supported by the host code reviewed in Section II-E, is architectural rather than algorithmic: every timestep requires a blocking host-to-device write of the spike array, three sequential kernel launches each individually waited on for profiling, and a blocking device-to-host read of the result, before the host can even begin preparing the next timestep's input. For a workload this small per-timestep (10,000 neurons, a few hundred thousand active synapses), PCIe transfer latency and kernel-launch overhead per timestep is comparable to or larger than the actual compute time, so the GPU's raw throughput advantage is never realized. This is a textbook instance of the latency-bound (rather than throughput-bound) GPU execution regime, and raises, though does not by itself prove, a conflict-avoidance or atomic-contention concern for GPU-mapped synaptic updates; verifying that specific claim would require the `propagate_kernel` source, which was not available for this report (see Section v.).

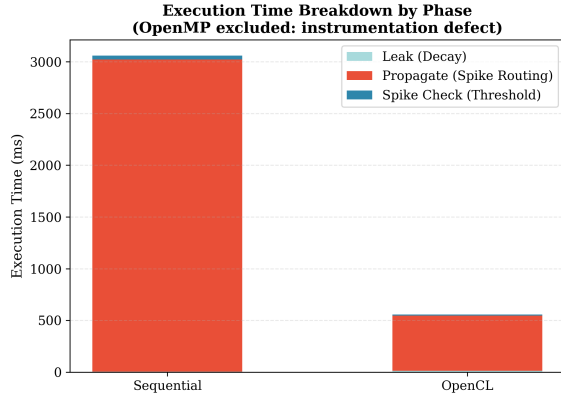


Fig. 6: Leak / propagate / spike-check time split by version, HPC node. OpenMP per-phase timings excluded due to instrumentation defect (see Section IV-D).

TABLE V: OpenCL Work-Group Size Sweep

Work-group size	Laptop Time (ms)	HPC Time (ms)
32	365.13	—
64	379.94	207.71
128	359.46	207.41
256	365.78	206.93
512	—	208.45

A caveat on Fig. 6: the per-phase timing breakdown collected by the OpenMP version shows internally inconsistent values (e.g., individual phase times reported in the billions of milliseconds) in the raw comparison.csv. Tracing the timing code in `snn_openmp.cpp` shows each thread keeps its own private `local_t0` timestamp, but the `#pragma omp single` blocks used to record phase boundaries can be executed by a different thread on each call, so a thread’s `local_t0` from a previous phase or iteration is sometimes read by the single block instead of the one actually intended; the result is occasional spurious deltas. This is a genuine instrumentation defect in the OpenMP version, not a result of misconfiguration, and the per-phase breakdown for OpenMP should be treated as unreliable until that timer is corrected (e.g., by making `local_t0` a shared, master-thread-only variable). The `total_time_ms` figures used everywhere else in this report come from the independent, single-threaded `Timer` class wrapping the whole `simulate()` call and are not affected by this bug.

A related note on Fig. 5/Table (Section 4.4 data): the HPC comparison.csv reports `synaptic_ops` of 3,577,342,869 for Sequential/OpenMP versus 3,968,263,680 for OpenCL, an approximately 11% difference, even though both runs report the identical `total_spikes` count (19,820,953). This gap is the same estimate-vs-exact-count metric difference identified in Section II-E and Section IV-A (OpenCL: spikes x average out-degree; Sequential/OpenMP: exact per-edge count), not evidence of a different connectivity matrix. Because the OpenCL SOPS figure (2.03 Giga-SOPS) is computed from this estimated count, it carries roughly the same $\pm 11\%$ uncertainty and should be read as approximate rather than as precisely comparable to the exact-count SOPS figures from the other two versions.

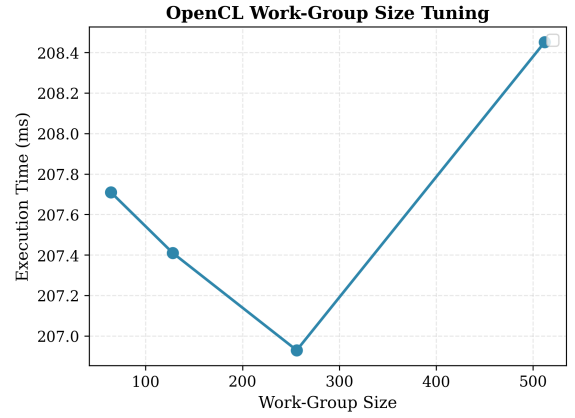


Fig. 7: Execution time vs. work-group size, HPC node. Times span only 1.5 ms across a $4\times$ range of work-group sizes, reflecting kernel-launch overhead on a near-idle network (10 spikes); no optimum can be identified. See Section IV-E.

E. OpenCL Work-Group Size Sweep

The HPC sweep’s raw `wg_sweep.csv` reports only 10 total spikes and 601 `synaptic_ops` across all four work-group sizes, for a 3,000-neuron, 300-timestep run. That is consistent with an almost entirely idle network, not one performing meaningful synaptic propagation, so the 206.93-208.45 ms timings on the HPC node are measuring kernel-launch and host-device synchronization overhead on a near-empty workload rather than the propagate kernel’s actual throughput. The HPC work-group-size conclusion is therefore withdrawn: it cannot be claimed that `wg=256` is optimal for this kernel when the kernel had essentially no work to do, and this experiment needs to be rerun on the HPC node with input parameters confirmed to produce sustained spiking activity (as the laptop run and the other HPC experiments in this report do) before any conclusion about optimal work-group size on that platform can be drawn.

The laptop sweep does not show this failure mode and is read at face value, with one caveat: work-group size 128 is fastest (359.46 ms), but the full sweep spans only about 20 ms (359.46-379.94 ms) across a $4\times$ range of work-group sizes. Given that this is a single run per configuration with no repeated trials (Section 5), a 20 ms spread is not strong evidence that 128 is meaningfully better than 32 or 256; it is reported as the best-observed configuration in this run, not as a statistically established optimum.

F. Roofline Model

The Roofline model plots achieved performance (Giga-SOPS) against Operational Intensity (synaptic operations per byte transferred), bounded by a horizontal compute-peak roof and a diagonal memory-bandwidth roof; their intersection (the ridge point) separates the compute-bound region from the memory-bound region [4].

Every measured point on both platforms falls far to the left of its ridge point, confirming the workload is unambiguously memory-bound on both a 2-core laptop and a 40-core/80-thread HPC node: scaling up the hardware does not change the qualitative diagnosis, only the size of the gap. The gap is in fact worse on the larger platform: laptop points achieve 16.6-17.8% of the memory-bandwidth ceiling, while HPC points achieve

TABLE VI: Roofline Model Results

Platform	Network Size	OI (ops/byte)	Performance (Giga-SOPS)	% of memory-BW ceiling
Laptop (ridge OI = 4.688)	1,000 neurons	0.0102	0.047	17.8%
Laptop	3,000 neurons	0.2457	1.076	17.1%
Laptop	5,000 neurons	0.2475	1.053	16.6%
HPC (ridge OI = 13.672)	10,000 neurons	0.2469	1.072	5.7%
HPC	20,000 neurons	0.2484	0.630	3.3%
HPC	30,000 neurons	0.2490	0.474	2.5%

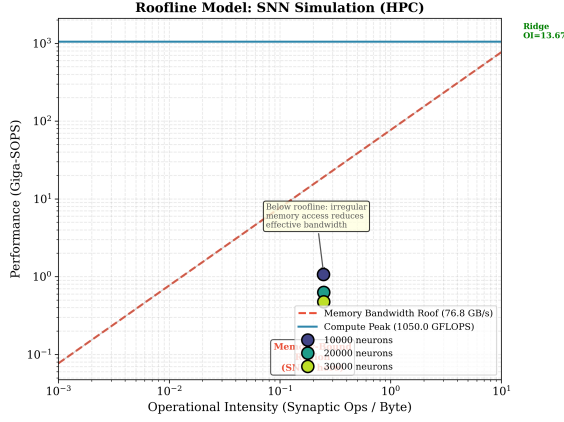


Fig. 8: Operational intensity vs. performance for both platforms, with memory-bandwidth and compute-peak roofs.

only 2.5-5.7%, and utilization falls monotonically as network size grows (10,000 to 30,000 neurons) on the HPC node. This is consistent with the propagate phase’s access pattern: each active synapse performs a single scattered, data-dependent write to $V[\text{col_idx}[\text{idx}]]$, an access pattern with essentially no spatial locality. As network size grows, the working set increasingly exceeds cache capacity (50 MB L3 on the HPC node vs. 3 MB on the laptop) and effective bandwidth utilization degrades further, exactly the trend the data shows. This pattern, low arithmetic intensity combined with poor spatial locality, is precisely the workload class that motivates non-von-Neumann neuromorphic hardware, discussed in Section V.

V. DISCUSSION AND FUTURE WORK

The consistent finding across both platforms, both scaling experiments, and the Roofline analysis is that SNN simulation performance on conventional CPU/GPU hardware is bounded by irregular memory access to the synaptic weight and potential arrays, not by raw compute throughput or even by core/thread count once a moderate thread count is reached. This is the software-level signature of what neuromorphic hardware research calls the memory wall: in a von Neumann machine, computation and the data it operates on are physically separated, so an operation as cheap as a single weighted accumulate ($V[\text{col_idx}] += \text{weight}$) still costs a full trip across the memory hierarchy whenever the target address is unpredictable, as it is here. Dedicated neuromorphic chips such as Intel Loihi [2] and IBM TrueNorth [3] address this directly by co-locating synaptic memory with the compute logic for each neuron core, so that a spike’s fan-out is resolved with local, fixed-latency, near-memory operations rather than scattered accesses into a shared, distant memory bank. The results in Section IV-F, particularly that increasing core count on the HPC node made the bandwidth-utilization problem worse rather than better, are

a direct, empirical argument for why this architectural shift, rather than simply adding more conventional cores, is necessary for SNN simulation at scale.

Limitations and threats to validity. Several aspects of this study should be qualified honestly rather than presented as more conclusive than the evidence supports. First, the OpenCL kernel source (`kernels.cl`) was not available for this report, so claims about the propagate kernel’s in-kernel conflict-resolution strategy (atomics vs. a race-free layout) could not be verified from code and are explicitly flagged wherever they arise; similarly, the OpenCL device actually used for the laptop runs (CPU runtime vs. integrated GPU) was not confirmed. Second, the OpenMP version’s per-phase timing instrumentation has a thread-attribution bug (Section 4.4) that invalidates the leak/propagate/spike-check breakdown for that version specifically, across all experiments that report it, not only the version-comparison table; total wall-clock times are unaffected. Third, the laptop and HPC strong/weak-scaling experiments use different problem sizes and densities (5,000 neurons at 5% density vs. 10,000 neurons at 2% density); the reason for this specific choice was not recorded beyond a general need to keep memory footprint tractable on each platform, and is reported here as an open item rather than a justified design decision. Fourth, the Roofline model’s bandwidth and compute-peak inputs are estimates, not independently measured figures, and the HPC figures supplied (76.8 GB/s, 1,050 GFLOPS) are closer to CPU-class than to the V100 accelerator’s published specifications; consequently, the Roofline analysis is best read as a platform-level (host-and-device-combined) estimate rather than a GPU-specific one, since the OpenCL path’s measured performance is more plausibly limited by host-device transfer latency (Section IV-D) than by the device’s own memory bandwidth. Fifth, all measurements in this report are single-run; no repeated trials or variance/confidence-interval reporting were performed, so small differences (e.g., the 20 ms spread in the laptop work-group sweep, Section 4.5) should not be read as statistically distinguishable. Sixth, the HPC weak-scaling spike-count jump (Section IV-C) and the HPC work-group sweep’s near-zero spike count (Section IV-E) indicate that those two specific experiments need to be rerun and verified before their numeric conclusions are relied upon; this report includes the raw figures for transparency but withdraws or qualifies the conclusions drawn from them rather than presenting them as established results. Finally, build configuration (compiler, optimization flags) for each binary was not captured and is listed as unknown rather than assumed (Section II-D), which is the most likely explanation for the HPC node’s 1-thread OpenMP run outperforming its own sequential baseline (Section IV-B).

Future work should: (1) obtain and review the OpenCL kernel source directly to confirm the conflict-resolution strategy and identify any avoidable global-memory coalescing or atomic-contention costs; (2) restructure the OpenCL host loop to keep the spike array resident on the device across multiple timesteps (e.g., generating Poisson input on-device, or batching several timesteps per host-device round trip), which is the single highest-impact fix available for the latency-bound behavior identified in Section IV-D and should be prioritized over other optimizations; (3) rerun the HPC weak-scaling and work-group-size experiments with the network’s spiking activity logged and confirmed sane at every configuration before reporting timing conclusions from them; (4) independently benchmark each platform’s real memory bandwidth (STREAM) and the V100’s real achievable bandwidth, to replace the estimated Roofline inputs with measured ones, ideally with separate CPU and GPU roofline ceilings; (5) fix the OpenMP phase-timing instrumentation bug; and (6) given that the binding constraint identified throughout this report is irregular memory access rather than thread or core count, prioritize memory-layout changes (e.g., a structure-of-arrays layout, blocking the potential array for cache reuse, or an alternative sparse format for the propagate phase) over further increases in thread count or a move to a different accelerator, since those layout changes target the bottleneck this report actually identifies.

VI. CONCLUSION

This project implemented and benchmarked sequential, OpenMP, and OpenCL versions of a Leaky Integrate-and-Fire spiking neural network on a 2-core laptop and a 40-core/80-thread HPC node with GPU acceleration. The central, well-supported finding is that performance is governed by irregular, scattered memory access to a sparse synaptic connectivity structure, not by available thread or core count: the OpenMP version’s best result (8.13x speedup at 16 threads) degrades at higher thread counts on the very platform with the most cores available, and Roofline analysis places every measured configuration on both platforms deep in the memory-bound region, with utilization of the memory-bandwidth ceiling actually worsening as the HPC network grows from 10,000 to 30,000 neurons. The OpenCL/GPU path underperformed the CPU-parallel path on both platforms, most plausibly due to per-timestep host-device synchronization rather than any property of the GPU’s compute or memory hardware. Several data-quality issues were identified during this analysis (an estimate-vs-exact-count metric mismatch in the OpenCL throughput figures, a probable network-instability artifact in part of the HPC weak-scaling sweep, and a near-idle network in the HPC work-group sweep) and are reported transparently with their numeric conclusions qualified or withdrawn accordingly, rather than presented as clean results. With those caveats stated, the empirical pattern, more cores and a discrete GPU both failing to convert into proportional throughput on a sparse, irregular workload, is consistent with, and helps motivate, the case for co-locating memory and compute at the neuron level in dedicated neuromorphic hardware such as Loihi and TrueNorth.

VII. REPRODUCIBILITY AND CODE DOCUMENTATION

All three implementations (`snn_sequential.cpp`, `snn_openmp.cpp`, `snn_opencl.cpp`) share a common header set (`types.hpp` for biological/simulation constants, `sparse_matrix.hpp` for the CSR connectivity structure, `utils.hpp` for timing and CLI argument parsing), ensuring a single, consistent definition of the LIF model and command-line interface across versions. The benchmark harness (`run_all_experiments.py`) compiles all three versions, runs the full correctness, strong-scaling, weak-scaling, version-comparison, and work-group-sweep experiment suite for a given platform (`--platform laptop|hpc`), and writes results to CSV. `plot_all.py` and `roofline.py` consume those CSVs to generate all eight figures referenced in this report. [<https://github.com/contactmevial/Neuromorphic-Workload-Acceleration.git>]

REFERENCES

- [1] W. Maass, “Networks of spiking neurons: The third generation of neural network models,” *Neural Networks*, vol. 10, no. 9, pp. 1659-1671, 1997.
- [2] M. Davies et al., “Loihi: A neuromorphic manycore processor with on-chip learning,” *IEEE Micro*, vol. 38, no. 1, pp. 82-99, 2018.
- [3] P. A. Merolla et al., “A million spiking-neuron integrated circuit with a scalable communication network and interface,” *Science*, vol. 345, no. 6197, pp. 668-673, 2014.
- [4] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65-76, 2009.
- [5] C. D. Schuman et al., “A survey of neuromorphic computing and neural networks in hardware,” arXiv preprint arXiv:1705.06963, 2017.
- [6] J. Bouvier et al., “Spiking neural networks hardware implementations and challenges: A survey,” *ACM Journal on Emerging Technologies in Computing Systems*, vol. 15, no. 2, pp. 1-35, 2019.
- [7] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, Version 5.2, 2021.
- [8] Khronos OpenCL Working Group, *The OpenCL Specification*, Version 1.2, 2012.